

Università degli Studi di Salerno

Corso di Ingegneria del Software

Easy Work Management System
UnitTest – CasesSpecification - TaskManagement
Versione 1.0



E.W.M.S.

Easy Work Management System

Data: 06/02/2026

Progetto: Easy Work Management System	Versione: 1.0
Documento: Problem statement	Data: --

Coordinatore del progetto:

Nome	Matricola
Giovanni Robustelli	0512121282
Vincenzo Mauro	0512119080

Partecipanti:

Nome	Matricola
Giovanni Robustelli	0512121282
Vincenzo Mauro	0512119080

Scritto da:	Vincenzo Mauro, Giovanni Robustelli
--------------------	-------------------------------------

Revision History

Data	Versione	Descrizione	Autore
06/02/2026	1.0	Uniti insieme le test cases specification indicate all'interno del documento UnitTest_TestCases_TaskManagement	Giovanni Robustelli Vincenzo Mauro

Indice

Introduzione	4
Classe: TaskCommonService	4
getTask	4
holdTask	5
Classe: TaskSupervisoreService	6
getAllTaskSup	7
createTask	8
getAllDipendentiInfo	9
deleteTask	10
Classe: TaskDipendenteService	10
getAllTaskDip	11
inizializzaTask	12
completaTask	13

Introduzione

In questo documento verranno inseriti le test cases specification del sottosistema “**TaskManagement**”, indicate nel documento **UnitTest_TestCases_TaskManagement_EWMS**.

Classe: TaskCommonService

In questa sezione analizziamo i test cases per il TaskCommonService definiti nel test plan.

Per facilità di lettura, le caselle con ‘//’ indicano una ripetizione della casella immediatamente superiore, inoltre le eccezioni nell’oracolo sono abbreviate in acronimi e i test frames verranno suddivisi per risultato atteso.

*(IllegalArgumentException) | **(IllegalStateException)

Precondizione classe di test:

- Creare mock per taskDAO, per simulare la risposta da parte del DAO alle invocazioni della classe sotto test.
- Creare mock per persistenceService: viene utilizzata per ottenere la corretta istanziazione del mock di taskDAO.

getTask

Test Case Id:	TCST_1
Precondizione:	Deve essere inizializzato un oggetto di tipo Task
Flusso degli eventi	

1. Vengono inseriti nel test unit i seguenti parametri, separati per test frame e risultato atteso

Test frame Id	Input (taskId)	Stato del sistema	Oracolo
TF_1	> 0	Presente nel DB	Successo: Ritorna l’oggetto Task

Test frame Id	Input (taskId)	Stato del sistema	Oracolo
TF_2	> 0	Non presente nel DB	Successo: Ritorna null

Test frame Id	Input (taskId)	Stato del sistema	Oracolo
TF_3	≤ 0	-	IAE*

holdTask

Test Case Id:	TCST 2
Precondizione:	Deve essere inizializzato un oggetto di tipo Task
Flusso degli eventi	

1. Vengono inseriti nel test unit i seguenti parametri, separati per test frame e risultato atteso

Test frame Id	Input (taskId)	Stato del DB	Stato DB (Status corrente)	Oracolo
TF_1	> 0	Presente nel DB	IN_ESECUZIONE	Successo: Ritorna true (lo stato diventa 'IN_SOSPENSIONE')

Test frame Id	Input (taskId)	Stato del DB	Stato DB (Status corrente)	Oracolo
TF_2	> 0	Presente nel DB	IN_SOSPENSIONE	Successo: Ritorna true (lo stato resta 'IN_SOSPENSIONE')

Test frame Id	Input (taskId)	Stato del DB	Stato DB (Status corrente)	Oracolo
TF_3	> 0	Assente nel DB	Irrilevante	Successo: Ritorna false (oggetto non trovato, nessuna operazione)

Test frame Id	Input (taskId)	Stato del DB	Stato DB (Status corrente)	Oracolo
TF_4	> 0	Presente nel DB	COMPLETATO / DA_COMPLETARE	ISE** (violazione pre-condizione)

Test frame Id	Input (taskId)	Stato del DB	Stato DB (Status corrente)	Oracolo
TF_5	≤ 0	Irrilevante	Irrilevante	IAE*

Classe: TaskSupervisoreService

Come prima, analizzeremo i test cases relativi al TaskSuperioreService, ovvero la parte che si occupa di gestire i task da un ruolo di supervisore.

*(IllegalArgumentException) | **(RuntimeException)

Precondizione classe di test:

- Creare mock per taskDAO, per simulare la risposta da parte del DAO alle invocazioni della classe sotto test.
- Creare mock per persistenceService: viene utilizzata per ottenere la corretta istanziazione del mock di taskDAO.

getAllTaskSup

Test Case Id:	TSST 1
Precondizione:	Deve essere istanziato un oggetto Supervisore e un oggetto Task
Flusso degli eventi	

1. Vengono inseriti nel test unit i seguenti parametri, separati per test frame e risultato atteso

Test frame Id	Input (Supervisore)	Dettagli input (ruolo/matricola)	Stato del DB (istanza Task)	Oracolo
TF_1	Istanziato	SUPERVISORE/ matr. > 0	Presenti	Successo: Restituisce la List<Task> popolata

Test frame Id	Input (Supervisore)	Dettagli input (ruolo/matricola)	Stato del DB (istanza Task)	Oracolo
TF_2	Istanziato	SUPERVISORE/ matr. > 0	Supervisore/Task inesistenti	Successo: Restituisce la List<Task> vuota

Test frame Id	Input (Supervisore)	Dettagli input (ruolo/matricola)	Stato del DB (istanza Task)	Oracolo
TF_3	[null]	Irrilevante	Irrilevante	IAE*
TF_4	Istanziato	DIPENDENTE/ matr. > 0	//	
TF_5	//	SUPERVISORE/ matr ≤ 0	//	

createTask

Test Case Id:	TSST 2
Precondizione:	Inizializzare in maniera corretta tutti i campi di inserimento della funzione createTask, creare mock UtenteDAO
Flusso degli eventi	

1. Vengono inseriti nel test unit i seguenti parametri, separati per test frame e risultato atteso

Test frame Id	Input (testo/enum)	Input (Date)	Input (matricole)	Stato del DB (FK)	Oracolo
TF_1	Validi	Scadenza futura	> 0	Entrambi esistono	Successo: Chiama DAO create

Test frame Id	Input (testo/enum)	Input (Date)	Input (matricole)	Stato del DB (FK)	Oracolo
TF_2	Titolo [null]	Valido	> 0	Irrilevante	IAE*
TF_3	Titolo vuoto	Valido	>0	//	
TF_4	Non validi (meno di 10 caratteri)	//	//	//	
TF_5	Validi	Scadenza < dataCreazione	//	//	
TF_6	//	Valido	SUPERVISORE ≤ 0	//	
TF_7	//	//	DIPENDENTE ≤ 0	//	
TF_8	//	//	//	Dipendente assente	
TF_9	//	//	//	Supervisore assente	
TF_10	Stato [null]	//	//	//	
TF_11	Priorità [null]	Valid	//	Irrilevante	

TF_12	Istruzioni [null]	Valid	//	//	
TF_13	Valid	Scadenza null	Positive	Irrilevante	

getAllDipendentiInfo

Test Case Id:	TSST 3
Precondizione:	Creare mock per la classe UtenteDAO
Flusso degli eventi	

1. Vengono inseriti nel test unit i seguenti parametri, separati per test frame e risultato atteso

Test frame Id	Input (matricola Sup.)	Stato del DB (Mock DAO)	Oracolo
TF_1	> 0	Restituisce la Lista Dipendenti	Successo: Ritorna List<Informazioni> popolata

Test frame Id	Input (matricola Sup.)	Stato del DB (Mock DAO)	Oracolo
TF_2	> 0 (non esiste)	Restituisce la Lista vuota	Successo: Ritorna List<Informazioni> vuota

Test frame Id	Input (matricola Sup.)	Stato del DB (Mock DAO)	Oracolo
TF_3	≤ 0	Irrilevante	IAE*

deleteTask

Test Case Id:	TSST 4
Precondizione:	
Flusso degli eventi	

1. Vengono inseriti nel test unit i seguenti parametri, separati per test frame e risultato atteso

Test frame Id	Input (taskId)	Stato del DB	Oracolo
TF_1	> 0	Presente	Successo: Ritorna true (task eliminato)

Test frame Id	Input (taskId)	Stato del DB	Oracolo
TF_2	> 0	Assente	Successo: Ritorna false

Test frame Id	Input (taskId)	Stato del DB	Oracolo
TF_3	≤ 0	Irrilevante	IAE*

Classe: TaskDipendenteService

Come prima, analizzeremo i test cases relativi al TaskDipendenteService, ovvero la parte che si occupa di gestire i task da un ruolo di supervisore.

*(IllegalArgumentException) | **(IllegalStateException)

Precondizione classe di test:

- Creare mock per taskDAO, per simulare la risposta da parte del DAO alle invocazioni della classe sotto test.
- Creare mock per persistenceService: viene utilizzata per ottenere la corretta istanziazione del mock di taskDAO.

getAllTaskDip

Test Case Id:	TPST 1
Precondizione:	Inizializzazione di un dipendente valido per i test frame positivi
Flusso degli eventi	

1. Vengono inseriti nel test unit i seguenti parametri, separati per test frame e risultato atteso

Test frame Id	Input (Dipendente)	Dettagli input (ruolo/matricola)	Stato del DB (istanza Task)	Oracolo
TF_1	Istanziato	DIEPNDEENTE/ matr. > 0	Presenti	Successo: Restituisce la List<Task> popolata

Test frame Id	Input (Dipendente)	Dettagli input (ruolo/matricola)	Stato del DB (istanza Task)	Oracolo
TF_2	Istanziato	DIPENDENTE/ matr. > 0	Assenti	Successo: Restituisce la List<Task> vuota

Test frame Id	Input (Dipendente)	Dettagli input (ruolo/matricola)	Stato del DB (istanza Task)	Oracolo
TF_3	[null]	Irrilevante	Irrilevante	IAE*
TF_4	Istanziato	SUPERVISORE/ matr. > 0	//	
TF_5	//	DIPENDENTE/ matr ≤ 0	//	

inizializzaTask

Test Case Id:	TPST 2
Precondizione:	Creazione di un task simulato (solo campi necessari) per inizializzare il task (return del mock)
Flusso degli eventi	

1. Vengono inseriti nel test unit i seguenti parametri, separati per test frame e risultato atteso

Test frame Id	Input (Dipendente)	Stato DB (esistenza)	Stato del DB (status corrente)	Oracolo
TF_1	> 0	Presente	DA_COMPLETARE	Successo: Restituisce true
TF_2	//	//	IN_SOSPENSIONE	
TF_3	//	//	IN_ESECUZIONE	

Test frame Id	Input (Dipendente)	Stato DB (esistenza)	Stato del DB (status corrente)	Oracolo
TF_4	> 0	Assente	Irrelevante	Successo: Restituisce false

Test frame Id	Input (Dipendente)	Stato DB (esistenza)	Stato del DB (status corrente)	Oracolo
TF_5	> 0	Presente	COMPLETATO	ISE**

Test frame Id	Input (Dipendente)	Stato DB (esistenza)	Stato del DB (status corrente)	Oracolo
TF_6	≤ 0	Irrelevante	Irrelevante	IAE*

completaTask

Test Case Id:	TPST_3
Precondizione:	Creazione di un task simulato (solo campi necessari) per completare il task (return del mock)
Flusso degli eventi	

1. Vengono inseriti nel test unit i seguenti parametri, separati per test frame e risultato atteso

Test frame Id	Input (taskId)	Stato DB (esistenza)	Stato del DB (status corrente)	Oracolo
TF_1	> 0	Presente	IN_ESECUZIONE	Successo: Restituisce true (stato diventa COMPLETATO)

Test frame Id	Input (taskId)	Stato DB (esistenza)	Stato del DB (status corrente)	Oracolo
TF_2	> 0	assente	Irrelevante	Successo: Restituisce false (nessuna modifica)

Test frame Id	Input (taskId)	Stato DB (esistenza)	Stato del DB (status corrente)	Oracolo
TF_3	> 0	Presente	COMPLETATO	ISE**

Test frame Id	Input (taskId)	Stato DB (esistenza)	Stato del DB (status corrente)	Oracolo
TF_4	≤ 0	Irrelevante	Irrelevante	IAE*